Modern App Development II – Project Report (Jan 2026)

1. Student Details

Name: Manoranjan Kumar

Roll Number: 24F2008014

Email: 24f2008014@ds.study.iitm.ac.in

About Me: I'm pursuing the IITM BS in Data Science and Applications full-time as a standalone degree. I joined in May 2024, and this is my last term in the diploma level. I also work as a freelance editor, proofreader, and typesetter on the side.

2. Project Details

Project Title: Placement Portal Application

Problem Statement: To build a web application that allows an admin (institute), companies, and students to interact with the system based on their roles. Companies can register and list new job positions which will receive student applications, then shortlist applicants and potentially make them placement offers. Students can view open positions, apply to them, track application history/status, and accept/reject placement offers. The admin can view all students, companies, positions, and placements and can whitelist/blacklist students, companies, and positions.

Approach: A single-page application using Vue 3 as the frontend framework to communicate with the backend via API. The build step uses Vite. The backend uses Flask, a WSGI framework, in combination with SQLAlchemy as the ORM and various Flask libraries for authentication, schema validation, caching, etc.

3. AI/LLM Declaration

No use of AI/LLMs.

4. Technologies and frameworks used

Frontend:

- **Vue 3** – frontend web framework
- **Vite** – build tool for building Vue
- **Pinia** – store library for Vue that allows sharing state across components
- **Vue Router** – client-side routing library for Vue SPAs
- **axios** – promise-based frontend HTTP client
- **Bootstrap 5** – frontend toolkit
- **bootstrap-icons** – open-source SVG icon library
- **jwt-decode** – decode JWT tokens

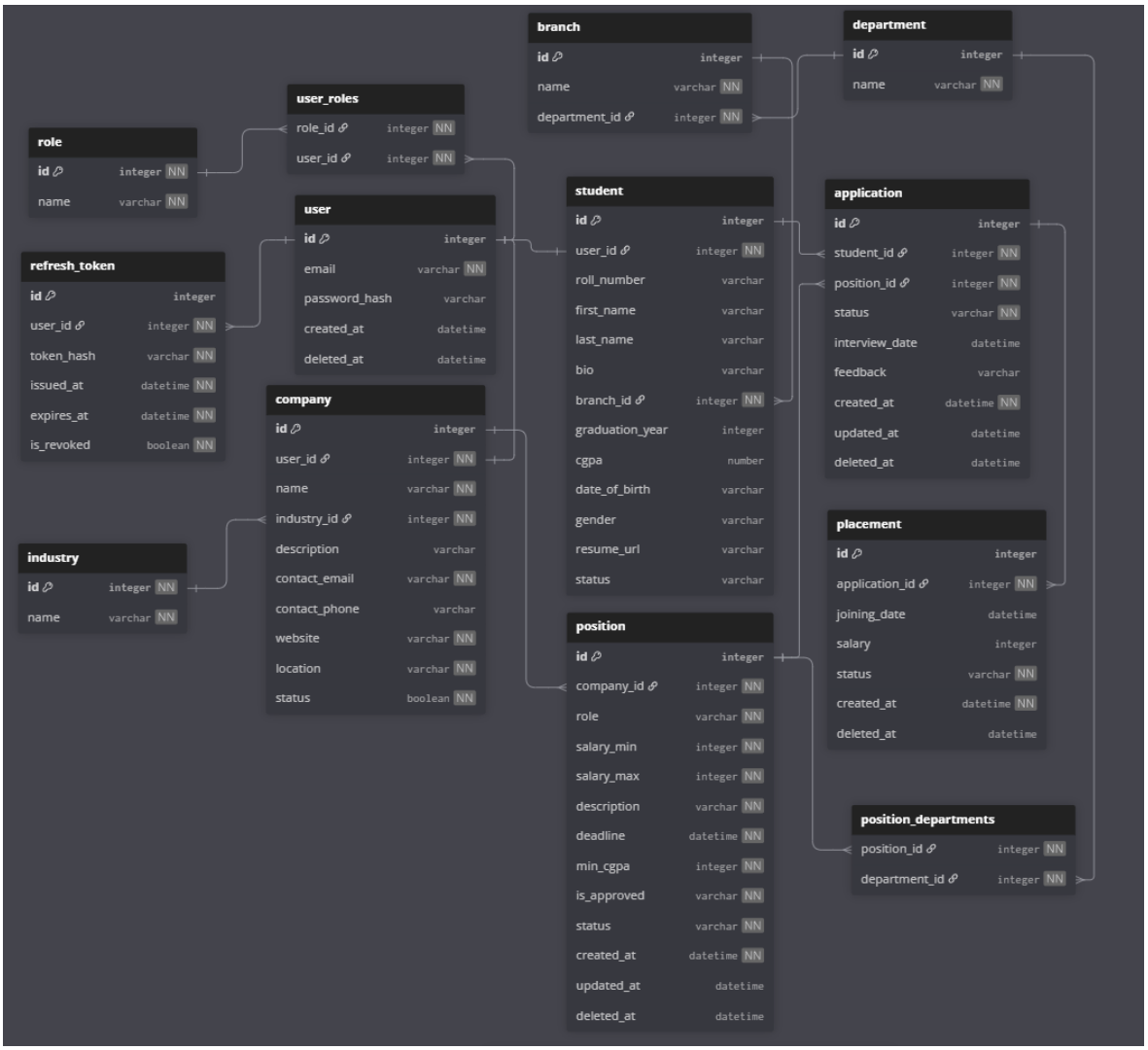
Backend:

- **Flask** – backend WSGI web app framework. Extensions used:
  - **Flask-Caching** – adds caching support (using **Redis** as the backend)
  - **Flask-CORS** – handles CORS on all Flask routes
  - **Flask-HTTPAuth** – provides basic, digest, and token HTTP authentication
  - **Flask-Mail** – sets up SMTP and allows sending messages/emails (with **Mailhog** for testing)
  - **Flask-Marshmallow** – integration layer between Flask and **marshmallow**
  - **Flask-Migrate** – handles SQLAlchemy DB migrations using **Alembic**

    - **Flask-SQLAlchemy** – adds SQLAlchemy support to Flask
- **APIFairy** – lightweight API framework that integrates marshmallow and auto-generates documentation
- **SQLAlchemy** – ORM library (using **SQLite** as the DB)
- **Celery** – task queueing and scheduling library
- **Marshmallow-OneOfSchema** – marshmallow extension to support polymorphic schemas
- **Marshmallow-SQLAlchemy** – SQLAlchemy integration with marshmallow
- **PyJWT** – allows encoding/decoding of JWTs
- **python-dotenv** – reads key-value pairs from an .env file and sets them as environmental variables
- **Redis** – in-memory key-value DB used as the cache backend
- **requests** – Python HTTP client library
- **tzdata** – contains time zone data

5. Database Schema / ER Diagram

Diagram (made using dbdiagram.io):



Tables:

- **Industry** – stores industry types (ex. Education, Technology)
- **Department** – stores departments (ex. Computer Science and Engineering)
- **Branch** – stores branches (ex. B.S. in Computer Science)
- **Role** – stores user role types (admin, company, student)
- **User** – stores user account details

- **User_Roles** – many-to-many table linking users and roles
- **RefreshToken** – stores users' refresh tokens
- **Company** – company profile details
- **Student** – student profile details
- **Position** – details of a position (job) listing
- **Position_Departments** – many-to-many table linking positions and eligible departments for that position
- **Application** – stores details of a student's application linked to a position
- **Placement** – stores details of a placement offer linked to a student's application

Relationships:

- **One-to-many:** User → RefreshToken, Department → Branch, Branch → Student, Company → Position, Student → Application, Position → Application, Application → Placement
- **Many-to-many:** User ⇔ Roles, Position ⇔ Departments
- **One-to-one:** User → Company, User → Student

6. API Resource Endpoints

- All endpoints are prefixed with **/api**. Ex. **/api/tokens/new**.
- Every endpoint except the token endpoints require bearer token authentication.
- Most endpoints that return lists of objects are filterable/sortable and accept query args.

Shared endpoints:

Endpoint	Method	Role(s)	Description
/home/stats	GET	All	Get landing page statistics
/tokens/new	POST	all	Authenticate user and return access token in body and refresh token in HttpOnly cookie (to log in)
/tokens/refresh	POST	all	Refresh an access token given a valid refresh token
/tokens/revoke	POST	all	Revoke an existing refresh token (to log out)
/users/me	GET	all	Get current user's account details
/users/me	PUT	all	Update user's account details
/register	POST	all	Register new user account
/departments	GET	all	Get all departments
/branches	GET	all	Get all branches
/industries	GET	all	Get all industries
/positions	GET	admin, student	Get all positions
/positions/:id	GET	all	Get position by ID
/positions/:id/applications	GET	admin, company	Get all applications for a position
/positions/:id/placements	GET	admin, company	Get all placements for a position
/companies/:id/positions	GET	admin, student	Get all positions for a company
/companies	GET	admin, student	Get all companies
/companies/:id	GET	admin, student	Get company by ID
/students/:id	GET	admin, company	Get student by ID
/applications/:id	GET	all	Get application by ID
/applications/:id	GET	all	Get placement by ID

/uploads/resumes/:filename	GET	all	Get student's resume by filename
/temp/:filename	GET	all	Get file by filename (for fetching generated CSV files)
/result/:id	GET	all	Get task status by ID

Admin-only endpoints:

Endpoint	Method	Description
/cache_test	GET	Test if cache is working
/stats	GET	Get admin dashboard statistics
/admin/monthly_report	GET	Manually trigger sending monthly admin report email
/students	GET	Get all students
/students/:id	PATCH	Update a student's status
/students/:id/applications	GET	Get a student's applications
/students/:id/placements	GET	Get a student's placements
/applications	GET	Get all applications
/placements	GET	Get all placements
/companies/:id	PATCH	Update a company's status
/positions/:id/whitelist	POST	Whitelist (approve) a position

Company-only endpoints:

Endpoint	Method	Description
/companies/me/stats	GET	Get company dashboard statistics
/companies/me	GET	Get your company's details
/companies/me	PUT	Update your company's details
/companies/me/positions	GET	Get your positions
/companies/me/applications	GET	Get applications for your positions
/companies/me/placements	GET	Get your placements
/positions	POST	Create new position
/positions/:id	PATCH	Update a position's status (open/closed)
/applications/:id/shortlist	POST	Shortlist an application for interview
/applications/:id/select	POST	Select an application and make a placement offer
/applications/:id/reject	POST	Reject an application with feedback

Student-only endpoints:

Endpoint	Method	Description
/students/me/stats	GET	Get student dashboard statistics
/students/me/reminder	GET	Manually trigger sending daily interview reminder email
/student/me/applications/export	GET	Export your applications as a .CSV file
/students/me	GET	Get your profile details
/students/me	PUT	Update your profile details
/students/me/applications	GET	Get your applications
/students/me/placements	GET	Get your placements
/students/me/positions/unapplied	GET	Get open positions you haven't applied for yet
/positions/:id/applications	POST	Create an application
/applications/:id/withdraw	POST	Withdraw a pending application
/placements/:id/accept	POST	Accept a placement offer
/placements/:id/reject	POST	Reject a placement offer

7. Architecture and Features

Architecture overview:

- **/client** – frontend
  - **/src**
    - **/assets** – public images
    - **/components** – each file is a Vue single-file component (.vue)
      - AccountEditForm, Alert, CompanyEditForm, CompanyRegisterForm, LoginForm, Modal, NavBar, PaginatedTable, PositionCreateForm, PositionViewForm, StudentEditForm, StudentRegisterForm
    - **/layouts/BaseLayout.vue** – the base layout template used for all pages
    - **/router/index.js** – Vue Router routes and router definitions and navigation guards
    - **/stores** – Pinia stores
      - **alert.js** – alert notification generation/deletion logic
      - **auth.js** – store current user details, set/delete/refresh/revoke access tokens
      - **tasks.js** – start/poll/stop polling long-running tasks
    - **/styles/style.css** – minor CSS tweaks
    - **/utils** – utility JS scripts
      - **api.js** – defines and configures axios instances and request/response interceptors for error handling
      - **helpers.js** – general helper functions for formatting dates, marking required form fields, downloading files, etc.
    - **/views** – views (all are .vue files)
      - AccountView, AdminDashboardView, ApplicationView, CompaniesView, CompanyDashboardView, CompanyPositionsView, CompanyView, HomeView, LoginView, PlacementsView, PlacementView, PositionsView, ProfileView, RegisterView, StudentDashboardView, StudentView
    - **App.vue** – root component of the Vue application
    - **main.js** – initializes and mounts Vue instance to index.html
  - **index.html** – entry point for the application where Vue is mounted
- **/server** – backend
  - **/api**
    - **/templates** – contains .html/.txt email templates
    - **admin.py** – admin API endpoints
    - **auth.py** – flask-httpauth basic and token auth methods and error handlers
    - **cli.py** – CLI commands for setting up the DB with user roles, the admin account, etc.
    - **company.py** – company API endpoints
    - **errors.py** – error handlers
    - **main.py** – shared API endpoints
    - **models.py** – SQLAlchemy model definitions
    - **schemas.py** – flask-marshmallow schema definitions
    - **student.py** – student API endpoints
    - **tasks.py** – Celery task definitions
    - **tokens.py** – token generation and verification functions and auth API endpoints
  - **/migrations** – flask-migrate DB version history
  - **application.py** – calls create_app() since I'm using the application factory pattern
  - **config.py** – config info and Celery task scheduling
  - **make_celery.py** – creates Celery instance

Features:

- User authentication using stateless JWTs and a combination of refresh and access tokens. Refresh tokens are stored on the user end in secure HttpOnly cookies.
- Paginated tables in frontend views allowing sorting, searching, and filtering by columns (including on joined columns).
- Frontend validation (JS/HTML form input validation) and backend validation (using marshmallow schemas), and error handling on both frontend (using axios error response interceptors and alert notifications) and backend (sending 4xx responses with descriptive error messages).
- Responsive frontend UI using Bootstrap 5.
- Student applications can be exported as .csv files, and daily reminders (for students' upcoming interviews) and monthly reminders (for the admin's monthly report) are both implemented using Celery.

8. Video Presentation

https://drive.google.com/file/d/1xnsO7bCdRUvsQbpJWqRdZFs44cenOQFq/view?usp=drive_link